

Ensamblador ARM instrucciones

Programas en assembler

Veamos cómo es un programa en assembler y qué partes tiene.

En el código hay una serie de elementos que aparecerán en todos los programas y que veremos a continuación

ejemplo00.asm



```
.data                                //Directiva de control: delimita sección de datos

.text                                // Directiva de control: delimita sección del código
.global main                         // visible en todo el programa
main:                               //etiqueta main, podría ser otra
    nop                             //esta inst no hace nada, consume un ciclo de reloj
fin:

    mov r7, #1                      //si R7=1 swi sabe que debe salir al S0
    swi 0                           //SWI, Software interrupt
```

Secciones de un programa en ensamblador

```
/* Comentario multilínea: nombre del programa y autor
*/
```

```
.data                //Directiva de control: delimita sección de datos
```

```
//----- Definición de datos
```

```
.text                // Directiva de control: delimita sección del código
```

```
.global main         // visible en todo el programa
```

```
main:                //etiqueta main, podría ser otra
```

```
    nop              //esta inst no hace nada, consume 1 ciclo de reloj
```

```
fin:
```

```
    mov r7, #1       // Se carga en R7 el syscall para swi, si R7=1 swi sabe que debe salir al SO
```

```
    swi 0             //SWI, Software interrupt
```

La sección de Datos: ejemplo00.asm

Tipos de datos de la sección .data

/*----- Definición de datos ----- */

ARM	Tipo en C	bits	Rango
.byte	unsigned char	8	0 a 255
	(signed) char	8	-128 a 127
.hword	unsigned short int	16	0 a 65.535
.short	(signed) short int	16	-32.768 a 32767
.word .int	unsigned int	32	0 a 4294967296
	(signed) int	32	-2147483648 a 2147483647
	unsigned long int	32	0 a 4294967296
	(signed) long int	32	-2147483648 a 2147483647
.quad	unsigned long long	64	0 a 2^{64}
	(signed) long long	64	-2^{63} a $2^{63}-1$

/*----- */

Tipos de datos básicos: directivas de asignación

.data

/----- Definición de datos: **Directivas de asignación** -----*/*

a1: **.byte 1** *// tipo byte, se inicializa en 1 si le agrego **.align** rellena*

var2: **.byte 'A'** *//tipo byte, caracter A*

var3: **.hword 25000** *// tipo hword 16 bits a 25000*

var4: **.word 0x12345678** *// tipo word de 32 bits*

b1: **.ascii "hola"** *//definición de cadena*

b2: **.asciz "mundo cruel"** *// definicion de cadena que termina con null*

dato1: **.zero 300** *// 300 bytes de valor cero*

dato2: **.space 200, 4** *// 200 bytes del valor 4*

.equ nada, 4 */* definimos constante nada con valor 4 tambien se puede valores negativos */*

.equ dividido, nada/2 *//dividido=2*

Instrucciones en ensamblador

- Instrucciones de transferencia de datos
- Instrucciones aritméticas
- Instrucciones de manejo de bits
- Instrucciones de transferencia de control
- Instrucciones lógicas

Instrucciones de transferencia de datos

mov{s}{cond} op1, op2

mov{cond} rd, #inmediato

{s} Es un sufijo opcional. Si se especifica, se actualiza el registro de flags (Registro CPSR) de acuerdo al resultado de la operación. (N Z C)

{cond} es condicional optativo, en general no lo usamos

#inmediato: es un valor en el rango 0-65535

mov r0, r1

mov r0, #2

Instrucciones aritméticas, suma

`add{s}{cond} {rd}, rn, Oper2`

- **s** es opcional, si se especifica, se actualiza el registro de flags CPSR, de acuerdo al resultado.
- **cond** opcional, es un código de condición.
- **rd** opcional. Registro destino
- **rn** es el registro que contiene el primer operando.
- **Oper2** segundo operando, no es siempre un registro

`add{cond} {rd}, rn, #imm12`

- **imm12** Es cualquier valor que va de 0-4095.

`adc{s}{cond} {rd}, rn, Oper2`

- **s** es opcional, si se especifica, se actualiza el registro de flags CPSR, de acuerdo al resultado.
- **cond** opcional, es un código de condición.
- **rd** opcional. Registro destino
- **rn** es el registro que contiene el primer operando.
- **Oper2** segundo operando, no es siempre un registro

`add r1,#20`

`adds r2,r5,r1`

`adc r1,r3`

Instrucciones aritméticas, resta

sub{s}{cond} {rd}, rn, Oper2

- **s** es opcional, si se especifica, se actualiza el registro de flags CPSR, de acuerdo al resultado.
- **cond** opcional, es un código de condición.
- **rd** opcional. Registro destino
- **rn** es el registro que contiene el primer operando.
- **Oper2** segundo operando, no es siempre un registro

sub{cond} {rd}, rn, #imm12

- **imm12** Es cualquier valor que va de 0-4095.

sbc{s}{cond} {rd}, rn, Oper2

- **s** es opcional, si se especifica, se actualiza el registro de flags CPSR, de acuerdo al resultado.
- **cond** opcional, es un código de condición.
- **rd** opcional. Registro destino
- **rn** es el registro que contiene el primer operando.
- **Oper2** segundo operando, no es siempre un registro

sub r1,#20

subs r2,r5,r1

sbc r1,r3